



Microprocessor & Computer Architecture (μpCA)

UE24CS251B

Session - 7

Microprocessor & Computer Architecture (μpCA)



UE24CS251B

**Multiple Load / Store Instructions or
Block Transfer**

MULTIPLE REGISTER LOAD / STORE

- Transfer a block of data more efficiently.
- Used for procedure entry and exit for saving and restoring workspace registers and the return address
- For ARM7, $2+Nt$ cycles
(N : #words, t : time for a word for sequential access).
- Increase interrupt latency since it can't be interrupted.
- Registers are considered in increasing order always.

LDMIA R1, {R0, R2, R5} @ R0 = mem[R1]

@ R2 = mem[R1+4]

@ R5 = mem[R1+8]

MULTIPLE REGISTER LOAD / STORE

Syntax: <LDM|STM>{<cond>}<addressing mode> $Rn\{!\}$,<registers>{^}

Addressing mode	Description	Start address	End address	$Rn!$
IA	increment after	Rn	$Rn + 4*N - 4$	$Rn + 4*N$
IB	increment before	$Rn + 4$	$Rn + 4*N$	$Rn + 4*N$
DA	decrement after	$Rn - 4*N + 4$	Rn	$Rn - 4*N$
DB	decrement before	$Rn - 4*N$	$Rn - 4$	$Rn - 4*N$

MULTIPLE REGISTER LOAD / STORE

LDM<mode> Rn, {<registers>}

IA: $\text{addr} := \text{Rn}$

IB: $\text{addr} := \text{Rn} + 4$

DA: $\text{addr} := \text{Rn} - \#<\text{registers}> * 4 + 4$

DB: $\text{addr} := \text{Rn} - \#<\text{registers}> * 4$

For each Ri in <registers>

IB: $\text{addr} := \text{addr} + 4$

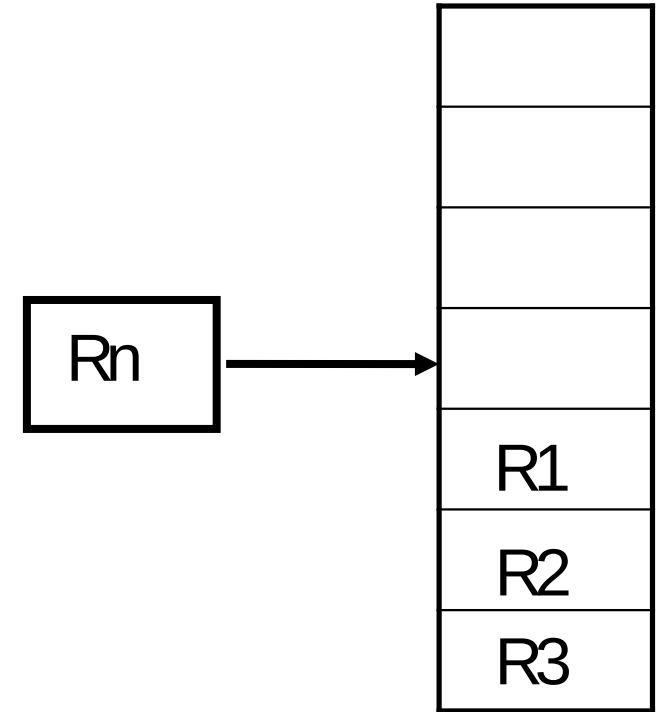
DB: $\text{addr} := \text{addr} - 4$

$\text{Ri} := \text{M}[\text{addr}]$

IA: $\text{addr} := \text{addr} + 4$

DA: $\text{addr} := \text{addr} - 4$

<!:>: $\text{Rn} := \text{addr}$



MULTIPLE REGISTER LOAD / STORE

LDMIA R0, {R1, R2, R3}

or

LDMIA R0, {R1-R3}

R1: 10

R2: 20

R3: 30

R0: 0x10



addr	data
0x010	10
0x014	20
0x018	30
0x01C	40
0x020	50
0x024	60

MULTIPLE REGISTER LOAD EXAMPLE- 01

- Program to transfer data from memory locations to registers using single **Load Multiple Instruction** using **IA condition**

. DATA

A: . WORD 100, 200, 300, 400, 500, 600, 700, 800

B: . WORD 0, 0, 0, 0, 0, 0, 0, 0

. TEXT

LDR R0, =A

LDR R7, =B

LDM A R0!, { R1, R8, R9, R11}

. END

MULTIPLE REGISTER STORE EXAMPLE- 01 CONTD. .

- Program to transfer data from memory locations to registers using single **STORE Multiple Instruction** using **IA condition**

. DATA

A: . WORD 100, 200, 300, 400, 500, 600, 700, 800

B: . WORD 0, 0, 0, 0, 0, 0, 0, 0

. TEXT

LDR R0, =A

LDR R7, =B

LDM A R0! , { R1, R8, R9, R11}

STM A R7! , { R1, R8, R9, R11}

. END

MULTIPLE REGISTER LOAD/STORE

LDM<mode> Rn, {<registers>}

IA: $\text{addr} := \text{Rn}$

IB: $\text{addr} := \text{Rn} + 4$

DA: $\text{addr} := \text{Rn} - \#<\text{registers}> * 4 + 4$

DB: $\text{addr} := \text{Rn} - \#<\text{registers}> * 4$

For each Ri in <registers>

IB: $\text{addr} := \text{addr} + 4$

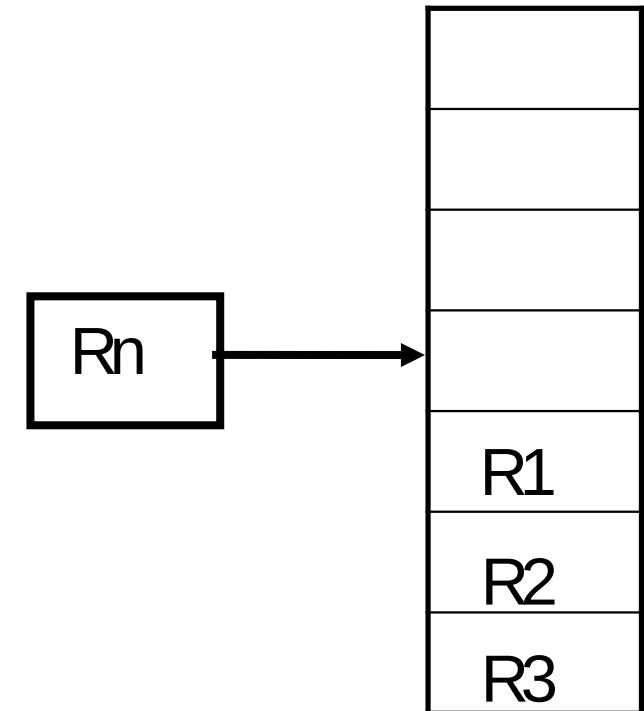
DB: $\text{addr} := \text{addr} - 4$

$\text{Ri} := \text{M}[\text{addr}]$

IA: $\text{addr} := \text{addr} + 4$

DA: $\text{addr} := \text{addr} - 4$

<!:>: $\text{Rn} := \text{addr}$



MULTIPLE REGISTER LOAD/STORE

LDMIB R0!, {R1, R2, R3}

R1: 20

R2: 30

R3: 40

R0: 0x01C



addr	data
0x010	10
0x014	20
0x018	30
0x01C	40
0x020	50
0x024	60

MULTIPLE REGISTER LOAD/ STORE EXAMPLE- 02

- Program to transfer data from memory locations to registers using

single **STORE Multiple Instruction using IB condition**
 . DATA

A: . WORD 100, 200, 300, 400, 500, 600, 700, 800, 900, 1000

B: . WORD 0, 0, 0, 0, 0, 0, 0, 0, 0, 0

. TEXT

LDR R0, =A

LDR R11, =B

ADD R0, R0, #- 4

ADD R11, R11, #- 4

LDM B R0! , { R1- R10}

STM B R11! , { R1- R10}

. END

MULTIPLE REGISTER LOAD/STORE

LDM $\langle \text{mode} \rangle$ R_n , {<register s>}

I A: $\text{addr} := R_n$

I B: $\text{addr} := R_n + 4$

DA: $\text{addr} := R_n - \# \langle \text{register s} \rangle * 4 + 4$

DB: $\text{addr} := R_n - \# \langle \text{register s} \rangle * 4$

For each R_i in <register s>

I B: $\text{addr} := \text{addr} + 4$

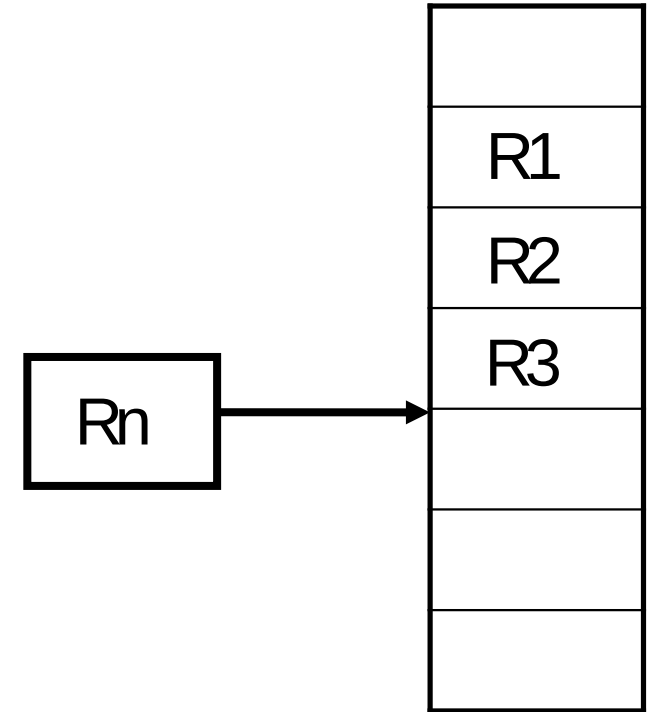
DB: $\text{addr} := \text{addr} - 4$

$R_i := M[\text{addr}]$

I A: $\text{addr} := \text{addr} + 4$

DA: $\text{addr} := \text{addr} - 4$

$\langle ! \rangle$: $R_n := \text{addr}$



MULTIPLE LOAD/STORE REGISTER

LDM**DA** R0!, {R1, R2, R3}

R1: 40

R2: 50

R3: 60

R0: 0x018



addr	data
0x010	10
0x014	20
0x018	30
0x01C	40
0x020	50
0x024	60

MULTIPLE REGISTER LOAD/STORE EXAMPLE- 02

- Program to transfer data from memory locations to registers using

single **STORE Multiple Instruction** using **DA condition**
. DATA

A: .WORD 100, 200, 300, 400, 500, 600, 700, 800, 900, 1000

B: .WORD 0, 0, 0, 0, 0, 0, 0, 0, 0, 0

. TEXT

LDR R0, =A

ADD R0, R0, #36

LDR R11, =B

ADD R11, R11, #36

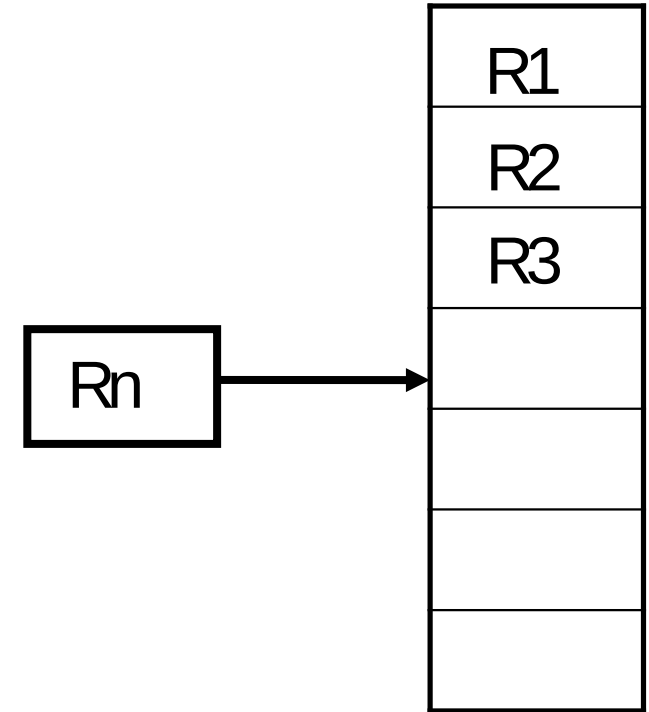
LDMDA R0!, { R1- R10}

STMDA R11!, { R1- R10}

. END

MULTIPLE LOAD/STORE REGISTER

LDM<mode> Rn, {<registers>}
IA: $\text{addr} := \text{Rn}$
IB: $\text{addr} := \text{Rn} + 4$
DA: $\text{addr} := \text{Rn} - \#<\text{registers}> * 4 + 4$
DB: $\text{addr} := \text{Rn} - \#<\text{registers}> * 4$
For each Ri in <registers>
 IB: $\text{addr} := \text{addr} + 4$
 DB: $\text{addr} := \text{addr} - 4$
 $\text{Ri} := \text{M}[\text{addr}]$
 IA: $\text{addr} := \text{addr} + 4$
 DA: $\text{addr} := \text{addr} - 4$
<!:>: $\text{Rn} := \text{addr}$



MULTIPLE LOAD/STORE REGISTER

LDMDB R0!, {R1, R2, R3}

R1: 30

R2: 40

R3: 50

R0: 0x018



addr	data
0x010	10
0x014	20
0x018	30
0x01C	40
0x020	50
0x024	60

MULTIPLE REGISTER LOAD/STORE EXAMPLE- 03

- Program to transfer data from memory locations to registers using single **STORE**

Multiple Instruction using DB condition

. DATA

A: .WORD 100, 200, 300, 400, 500, 600, 700, 800, 900, 1000

B: .WORD 0, 0, 0, 0, 0, 0, 0, 0, 0, 0

. TEXT

LDR R0, =A

ADD R0, R0, #40

LDR R11, =B

ADD R11, R11, #40

LDMDB R0!, { R1- R10}

STMDB R11!, { R1- R10}

. END

MULTIPLE REGISTER LOAD/STORE EXAMPLE- 04

PRE mem32[0x80018] = 0x03
 mem32[0x80014] = 0x02
 mem32[0x80010] = 0x01
 r0 = 0x00080010
 r1 = 0x00000000
 r2 = 0x00000000
 r3 = 0x00000000

 LDMIA r0!, {r1-r3}

POST r0 = 0x0008001c
 r1 = 0x00000001
 r2 = 0x00000002
 r3 = 0x00000003

MULTIPLE REGISTER LOAD/STORE EXAMPLE- 04

Address pointer	Memory		
	address	Data	
	0x80020	0x00000005	
	0x8001c	0x00000004	
	0x80018	0x00000003	$r3 = 0x00000000$
	0x80014	0x00000002	$r2 = 0x00000000$
$r0 = 0x80010 \rightarrow$	0x80010	0x00000001	$r1 = 0x00000000$
	0x8000c	0x00000000	

LDMIA $r0!, \{r1-r3\}$

Address pointer	Memory		
	address	Data	
	0x80020	0x00000005	
$r0 = 0x8001c \rightarrow$	0x8001c	0x00000004	
	0x80018	0x00000003	$r3 = 0x00000003$
	0x80014	0x00000002	$r2 = 0x00000002$
	0x80010	0x00000001	$r1 = 0x00000001$
	0x8000c	0x00000000	

MULTIPLE REGISTER LOAD/STORE EXAMPLE- 05

Address pointer	Memory address	Data	
	0x80020	0x00000005	
	0x8001c	0x00000004	
	0x80018	0x00000003	$r3 = 0x00000000$
	0x80014	0x00000002	$r2 = 0x00000000$
$r0 = 0x80010 \rightarrow$	0x80010	0x00000001	$r1 = 0x00000000$
	0x8000c	0x00000000	

LDMIB $r0!, \{r1-r3\}$

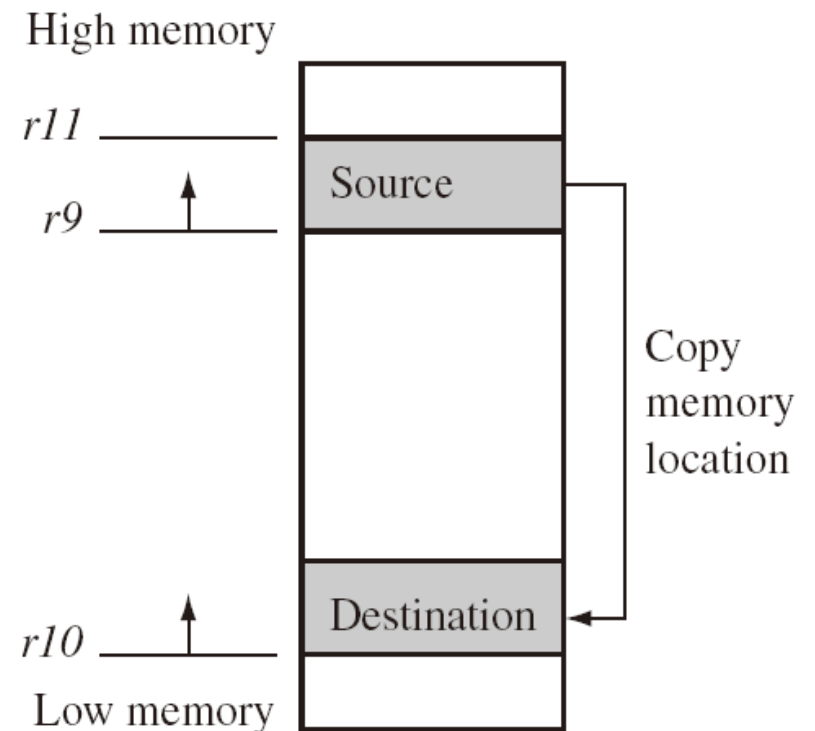
Address pointer	Memory address	Data	
	0x80020	0x00000005	
$r0 = 0x8001c \rightarrow$	0x8001c	0x00000004	$r3 = 0x00000004$
	0x80018	0x00000003	$r2 = 0x00000003$
	0x80014	0x00000002	$r1 = 0x00000002$
	0x80010	0x00000001	
	0x8000c	0x00000000	

MULTIPLE REGISTER LOAD / STORE- APPLICATION

Write a program using ARM/TDM - ISA to copy a block 512 bytes of data from location A to location B if the rate of data transfer rate is 32 bytes.

- R9: address of the source A
- R10: address of the destination B
- R11: count : 16

```
loop: ADD R11,R11,#1
      LDMIA R9!, {R0-R7}
      STMIA R10!, {R0-R7}
      CMP R11,#16
      BNE loop
```



Microprocessor & Computer Architecture (μpCA)



UE24CS251B

**Block Transfer
Instructions:**

Stack

Operations

Block Transfer Instructions(STACK)

The Memory access can be in LIFO order.

i.e Can be treated like STACK.

R13 is a stack pointer which will keep the address of TOP of the STACK.

Mainly used in Procedural Call.

Stack can grow upward or downward direction based on the MODE used by the user in the program

Addr	data
0x1010	
0x1014	
0x1018	10
0x101C	20
0x1020	30
0x1024	40
0x1028	50
0x102C	60
0x1030	
0x1034	
0x1038	

Block Transfer Instructions (STACK)

10

20

10

140

15

60

27

R13=0x1018

SP

Addr	data
0x1010	
0x1014	
0x1018	
0x101C	
0x1020	
0x1024	
0x1028	
0x102C	
0x1030	
0x1034	
0x1038	

Block Transfer Instructions (STACK)

10 20 75 140 15 60 27

R13=0x101C

SP

Addr	data
0x1010	
0x1014	
0x1018	10
0x101C	
0x1020	
0x1024	
0x1028	
0x102C	
0x1030	
0x1034	
0x1038	

Block Transfer Instructions (STACK)

10 20 75 140 15 60 27

R13=0x1020

SP



Addr	data
0x1010	
0x1014	
0x1018	10
0x101C	20
0x1020	
0x1024	
0x1028	
0x102C	
0x1030	
0x1034	
0x1038	

Block Transfer Instructions (STACK)

10 20 75 140 15 60 27

R13=0x1024

SP

Addr	data
0x1010	
0x1014	
0x1018	10
0x101C	20
0x1020	75
0x1024	
0x1028	
0x102C	
0x1030	
0x1034	
0x1038	

Block Transfer Instructions (STACK)

10 20 75 140 15 60 27

R13=0x1028

Addr	data
0x1010	
0x1014	
0x1018	10
0x101C	20
0x1020	75
0x1024	140
0x1028	
0x102C	
0x1030	
0x1034	
0x1038	

SP →

Block Transfer Instructions(STACK)

Syntax:

<LDM STM> <Addressing Mode> R13 {!},

Registers

Addressing Mode	=LDM	=STM
Full ascending (FA)	LMDA	STMIB
Full descending (FD)	LDMIA	STMDB
Empty ascending (EA)	LDMDB	STMIA
Empty descending (ED)	LDMIB	STMDA

- **STM** is used to **PUSH** on to the STACK
- **LDM** is used to **POP** from the STACK

Block Transfer Instructions (STACK)

Hexadecimal	
Unsigned Decimal	
Signed Decimal	
R0	: 00000000
R1	: 00000000
R2	: 00000000
R3	: 00000000
R4	: 00000000
R5	: 00000000
R6	: 00000000
R7	: 00000000
R8	: 00000000
R9	: 00000000
R10 (sl)	: 00000000
R11 (fp)	: 00000000
R12 (ip)	: 00000000
R13 (sp)	: 00005400
R14 (lr)	: 00000000
R15 (pc)	: 00001000

CPSR Register	
Negative (N)	: 0
Zero (Z)	: 0
Carry (C)	: 0
Overflow (V)	: 0
IRQ Disable	: 1
FIQ Disable	: 1
Thumb (T)	: 0
CPU Mode	: System

0x000000df	

```
.text
MOV R0, #4
MOV R1, #5
STMEA R13!, { R0, R1}
.end
```

Initially Stack is
Empty

StackView	
000053C4	: 81818181
000053C8	: 81818181
000053CC	: 81818181
000053D0	: 81818181
000053D4	: 81818181
000053D8	: 81818181
000053DC	: 81818181
000053E0	: 81818181
000053E4	: 81818181
000053E8	: 81818181
000053EC	: 81818181
000053F0	: 81818181
000053F4	: 81818181
000053F8	: 81818181
000053FC	: 81818181
00005400	: 81818181
00005404	: 81818181
00005408	: 81818181
0000540C	: 81818181
00005410	: 81818181
00005414	: 81818181
00005418	: 81818181
0000541C	: 81818181
00005420	: 81818181
00005424	: 81818181
00005428	: 81818181
0000542C	: 81818181
00005430	: 81818181
00005434	: 81818181
00005438	: 81818181
0000543C	: 81818181

Block Transfer Instructions (STACK)

RegistersView	
General Purpose Floating Point	
Hexadecimal	
Unsigned Decimal	
Signed Decimal	
R0	: 00000004
R1	: 00000005
R2	: 00000000
R3	: 00000000
R4	: 00000000
R5	: 00000000
R6	: 00000000
R7	: 00000000
R8	: 00000000
R9	: 00000000
R10 (s1)	: 00000000
R11 (fp)	: 00000000
R12 (ip)	: 00000000
R13 (sp)	: 00005408
R14 (lr)	: 00000000
R15 (pc)	: 00011400

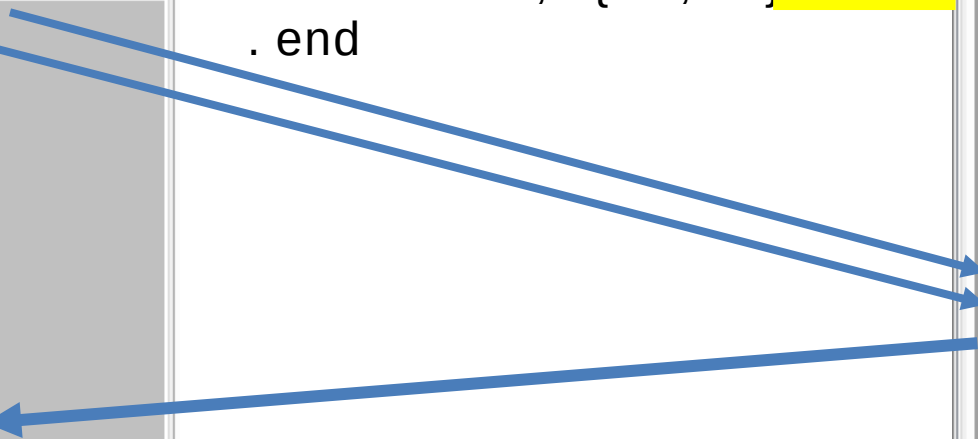
CPSR Register	
Negative (N)	: 0
Zero (Z)	: 0
Carry (C)	: 0
Overflow (V)	: 0
IRQ Disable	: 1
FIQ Disable	: 1
Thumb (T)	: 0
CPU Mode	: System

```

.text
MOV R0, #4
MOV R1, #5
STMEA R13!, { R0, R1 } PUSH
.end

```

StackView	
000053CC	: 81818181
000053D0	: 81818181
000053D4	: 81818181
000053D8	: 81818181
000053DC	: 81818181
000053E0	: 81818181
000053E4	: 81818181
000053E8	: 81818181
000053EC	: 81818181
000053F0	: 81818181
000053F4	: 81818181
000053F8	: 81818181
000053FC	: 81818181
00005400	: 00000004
00005404	: 00000005
00005408	: 81818181
0000540C	: 81818181
00005410	: 81818181
00005414	: 81818181
00005418	: 81818181
0000541C	: 81818181
00005420	: 81818181
00005424	: 81818181
00005428	: 81818181
0000542C	: 81818181
00005430	: 81818181
00005434	: 81818181
00005438	: 81818181
0000543C	: 81818181
00005440	: 81818181
00005444	: 81818181



Block Transfer Instructions (STACK)

RegistersView	
General Purpose Floating Point	
Hexadecimal	
Unsigned Decimal	
Signed Decimal	
R0	: 00000004
R1	: 00000005
R2	: 00000000
R3	: 00000000
R4	: 00000000
R5	: 00000004
R6	: 00000005
R7	: 00000000
R8	: 00000000
R9	: 00000000
R10 (sl)	: 00000000
R11 (fp)	: 00000000
R12 (ip)	: 00000000
R13 (sp)	: 00005400
R14 (lr)	: 00000000
R15 (pc)	: 00001010

CPSR Register	
Negative (N)	: 0
Zero (Z)	: 0
Carry (C)	: 0
Overflow (V)	: 0
IRQ Disable	: 1
FIQ Disable	: 1
Thumb (T)	: 0
CPU Mode	: System

0x000000df	

```
.text
MOV R0, #4
MOV R1, #5
STMEA R13!, { R0, R1}
LDMEA R13!, { R5, R6}
.end
```

POP

Again the Stack is Empty

StackView	
000053C4	: 81818181
000053C8	: 81818181
000053CC	: 81818181
000053D0	: 81818181
000053D4	: 81818181
000053D8	: 81818181
000053DC	: 81818181
000053E0	: 81818181
000053E4	: 81818181
000053E8	: 81818181
000053EC	: 81818181
000053F0	: 81818181
000053F4	: 81818181
000053F8	: 81818181
000053FC	: 81818181
00005400	: 00000004
00005404	: 00000005
00005408	: 81818181
0000540C	: 81818181
00005410	: 81818181
00005414	: 81818181
00005418	: 81818181
0000541C	: 81818181
00005420	: 81818181
00005424	: 81818181
00005428	: 81818181
0000542C	: 81818181
00005430	: 81818181
00005434	: 81818181
00005438	: 81818181
0000543C	: 81818181

1. In a "Full Descending" (FD) stack, which Load/Store Multiple instructions are used for POP and PUSH operations?

- A) POP = LDMDA, PUSH = STMIB
- B) POP = LDMIA, PUSH = STMDB
- C) POP = LDMDB, PUSH = STMIA
- D) POP = LDMIB, PUSH = STMDA

2. If R0 = 0x1000 and instruction is LDMDB R0!, {R1,R2,R3}, final value of R0 becomes:

- A) 0x1000
- B) 0x1004
- C) 0x0FF4
- D) 0x0FF0

1. In a "Full Descending" (FD) stack, which Load/Store Multiple instructions are used for POP and PUSH operations?

- A) POP = LDMDA, PUSH = STMIB
- B) POP = LDMIA, PUSH = STMDB**
- C) POP = LDMDB, PUSH = STMIA
- D) POP = LDMIB, PUSH = STMDA

2. If R0 = 0x1000 and instruction is LDMDB R0!, {R1,R2,R3}, final value of R0 becomes:

- A) 0x1000
- B) 0x1004
- C) 0x0FF4**
- D) 0x0FF0

3. If 512 bytes are copied using {R0-R7} in each iteration, number of loop iterations is:

- A) 8
- B) 16
- C) 32
- D) 64

4. Which addressing mode gives highest memory address first?

- A) IA
- B) IB
- C) DA
- D) DB

3. If 512 bytes are copied using {R0-R7} in each iteration, number of loop iterations is:

- A) 8
- B) 16**
- C) 32
- D) 64

4. Which addressing mode gives highest memory address first?

- A) IA
- B) IB
- C) DA**
- D) DB

5. Why does block transfer increase interrupt latency?

- A) Too many instructions
- B) Uses stack
- C) Cannot be interrupted mid-execution
- D) Uses pipeline

6. Which addressing mode accesses lowest memory address first?

- A) IA
- B) IB
- C) DA
- D) DB

5. Why does block transfer increase interrupt latency?

- A) Too many instructions
- B) Uses stack
- C) Cannot be interrupted mid-execution**
- D) Uses pipeline

6. Which addressing mode accesses lowest memory address first?

- A) IA**
- B) IB
- C) DA
- D) DB



THANK YOU



Team MPCA

Department of Computer Science
and Engineering